

LangChain Partners

集成层源码架构分析文档

核心流程图 | 时序图 | 类设计说明

基于 LangChain v0.1.16 源码分析

涵盖 OpenAI / Anthropic / Ollama / DeepSeek / HuggingFace
Groq / MistralAI / Fireworks / Chroma / Qdrant 等 15 个集成包

2026 年 3 月

目 录

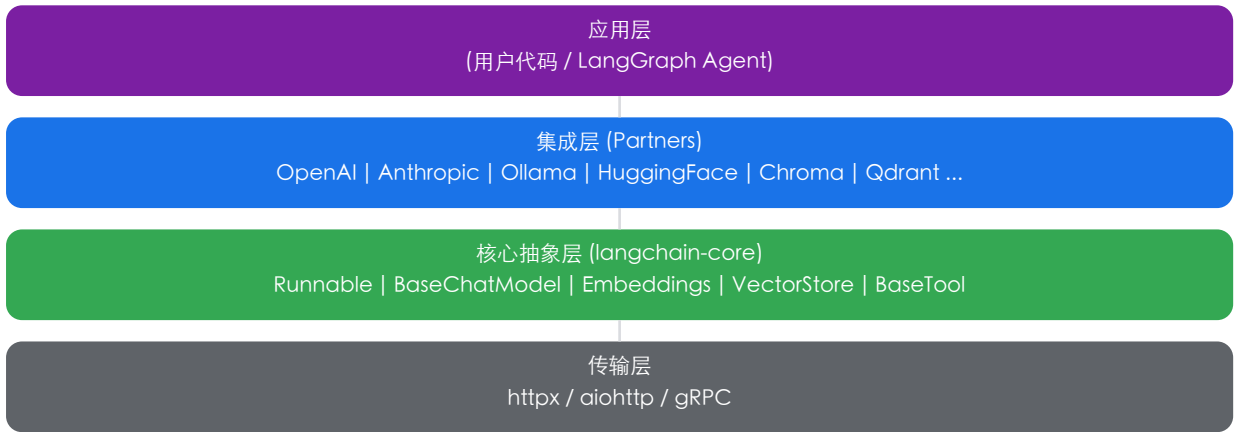
- 第一章 Partners 集成层整体架构概览
- 第二章 核心类继承体系设计
- 第三章 Chat Model 调用流程图
- 第四章 流式输出 (Streaming) 时序图
- 第五章 Tool Calling (工具调用) 流程图
- 第六章 结构化输出 (Structured Output) 流程图
- 第七章 Embeddings 嵌入处理流程图
- 第八章 向量存储 (VectorStore) 架构
- 第九章 OpenAI 集成包核心设计
- 第十章 Anthropic 集成包核心设计
- 第十一章 其他 Partner 集成包设计
- 第十二章 跨 Provider 对比分析

第一章 Partners 集成层整体架构概览

1.1 架构定位

LangChain Partners 是 LangChain 生态系统中的集成层，位于 langchain-core 基础抽象层之上，负责将各种第三方 AI 服务（大语言模型、嵌入模型、向量数据库等）接入 LangChain 统一的 Runnable 执行框架。每个 Partner 包都是独立版本管理的 Python 包，通过实现 langchain-core 定义的抽象接口来提供具体的服务集成。

1.2 分层架构图



1.3 Partner 包总览

包名	类型	核心类	继承方式
langchain-openai	LLM / Chat / Embed	ChatOpenAI, OpenAIEmbeddings	BaseChatOpenAI -> BaseChatModel
langchain-anthropic	Chat	ChatAnthropic	BaseChatModel (直接)
langchain-ollama	LLM / Chat / Embed	ChatOllama, OllamaEmbeddings	BaseChatModel (直接)
langchain-deepseek	Chat	ChatDeepSeek	BaseChatOpenAI (复用 OpenAI)
langchain-huggingface	LLM / Chat / Embed	ChatHuggingFace, HuggingFaceEmbeddings	BaseChatModel (直接)

包名	类型	核心类	继承方式
langchain-groq	Chat	ChatGroq	BaseChatModel (独立实现)
langchain-mistralai	Chat / Embed	ChatMistralAI, MistralAIEmbeddings	BaseChatModel (独立实现)
langchain-fireworks	LLM / Chat / Embed	ChatFireworks, FireworksEmbeddings	BaseChatModel (独立实现)
langchain-chroma	VectorStore	Chroma	VectorStore
langchain-qdrant	VectorStore	QdrantVectorStore	VectorStore

第二章 核心类继承体系设计

2.1 Runnable 统一执行接口

Runnable 是 LangChain 表达式语言（LCEL）的核心接口，所有可组合的组件都必须实现此协议。它定义了 `invoke`、`stream`、`batch` 三套方法（各含同步和异步版本），使得所有组件可以用运算符自由组合成链式管道。

2.2 完整类继承树

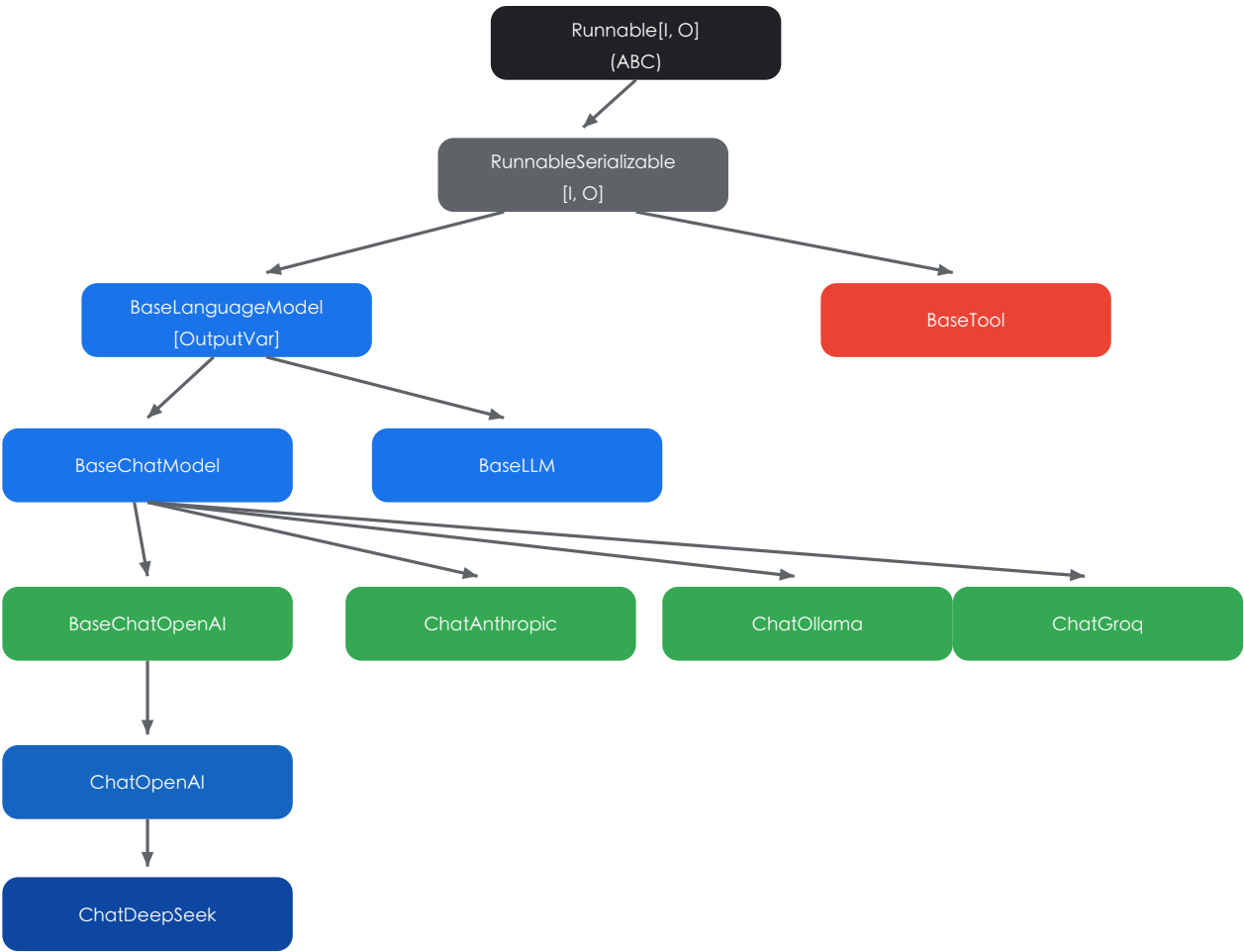


图 2-1: Partners 核心类继承关系图（绿色=Partner实现，蓝色=OpenAI系列）

2.3 独立接口类

注意：Embeddings 和 VectorStore 是独立的 ABC 抽象类，不继承 Runnable。它们保持纯净的功能接口，VectorStore 通过 as_retriever() 方法桥接到 Runnable 体系。

基类	继承	抽象方法	Partner 实现
Embeddings	ABC	embed_documents(), embed_query()	OpenAIEmbeddings, OllamaEmbeddings, HuggingFaceEmbeddings
VectorStore	ABC	similarity_search()	Chroma, QdrantVectorStore

2.4 三级实现模式

LangChain 采用三级实现模式：基类提供完整的公开 API (invoke/stream/batch)，但只要求子类实现带下划线的内部方法。基类负责回调管理、缓存、重试、速率限制等横切关注点。

层级	方法	职责	示例
公开 API	invoke() / stream() / batch()	回调链、缓存、重试、类型转换	BaseChatModel.invoke()
内部生成	_generate() / _stream()	实际的 API 调用和响应解析	ChatOpenAI._generate()
Payload 构建	_get_request_payload()	消息格式转换、参数组装	ChatAnthropic._get_request_payload()

第三章 Chat Model 调用流程图

3.1 统一调用流程

所有 Chat Model 的调用都遵循相同的流程，从用户调用 `invoke()` 开始，经过基类的回调和缓存管理，最终调用 Partner 包的 `_generate()` 方法完成实际的 API 请求。

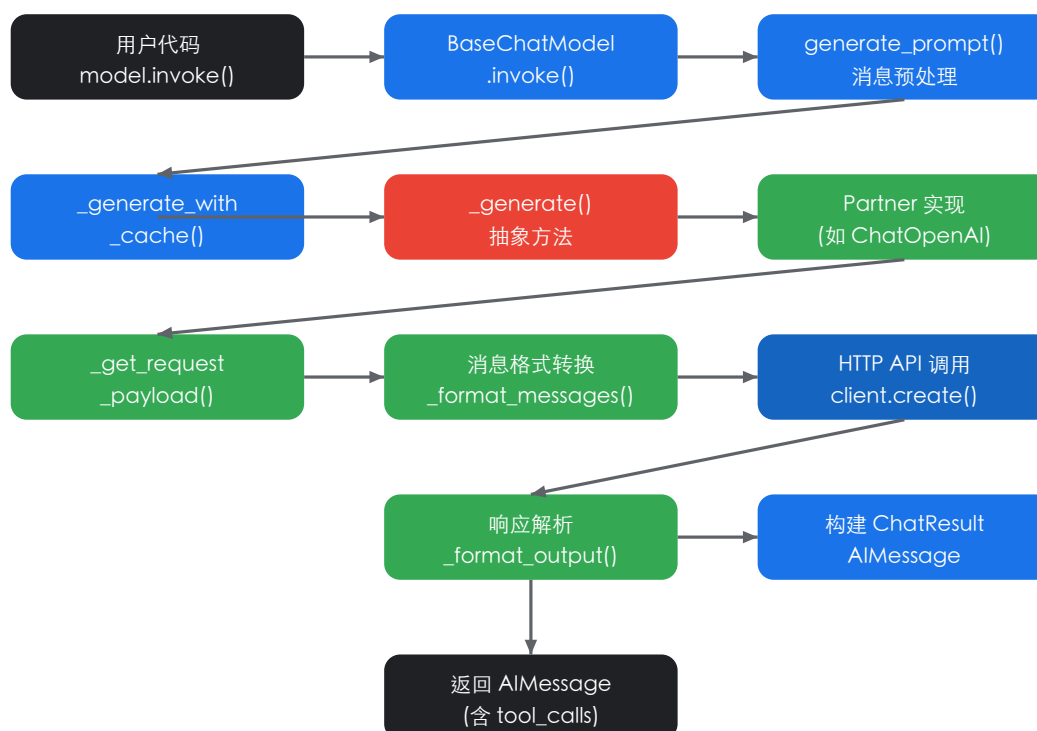


图 3-1: Chat Model 统一调用流程图

3.2 OpenAI 双 API 路由机制

`ChatOpenAI` 是唯一同时支持 Chat Completions API 和 Responses API 的实现。通过 `_use_responses_api()` 方法自动检测应使用哪个 API：

- 使用了内置工具 (`web_search`, `file_search`, `computer_use` 等)
- 使用了 Responses API 专属参数 (`reasoning`, `truncation`, `previous_response_id`)
- `output_version` 设置为 'responses/v1'
- 模型名称匹配 Responses API 专属前缀 (如 `gpt-5-pro`)

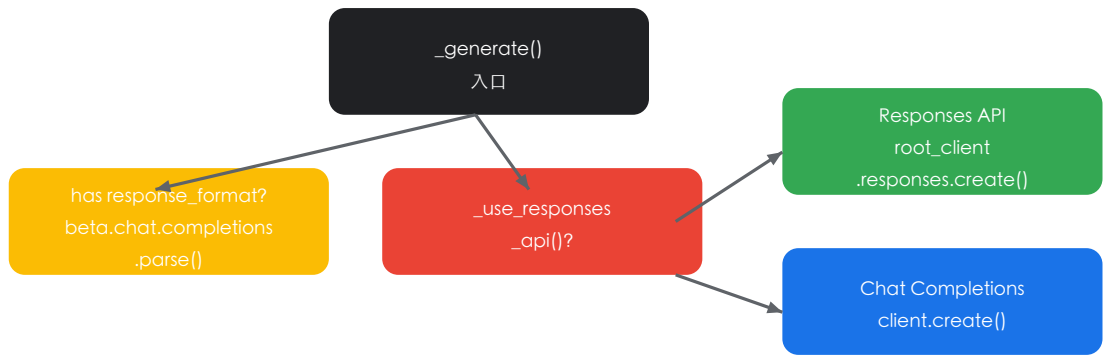


图 3-2: OpenAI 双 API 路由决策流程

第四章 流式输出 (Streaming) 时序图

4.1 OpenAI Chat Completions 流式时序

流式输出是 LangChain 中最常用的交互模式。当调用 `stream()` 时，系统会逐块返回 `AIMessageChunk`，允许前端实时展示生成内容。

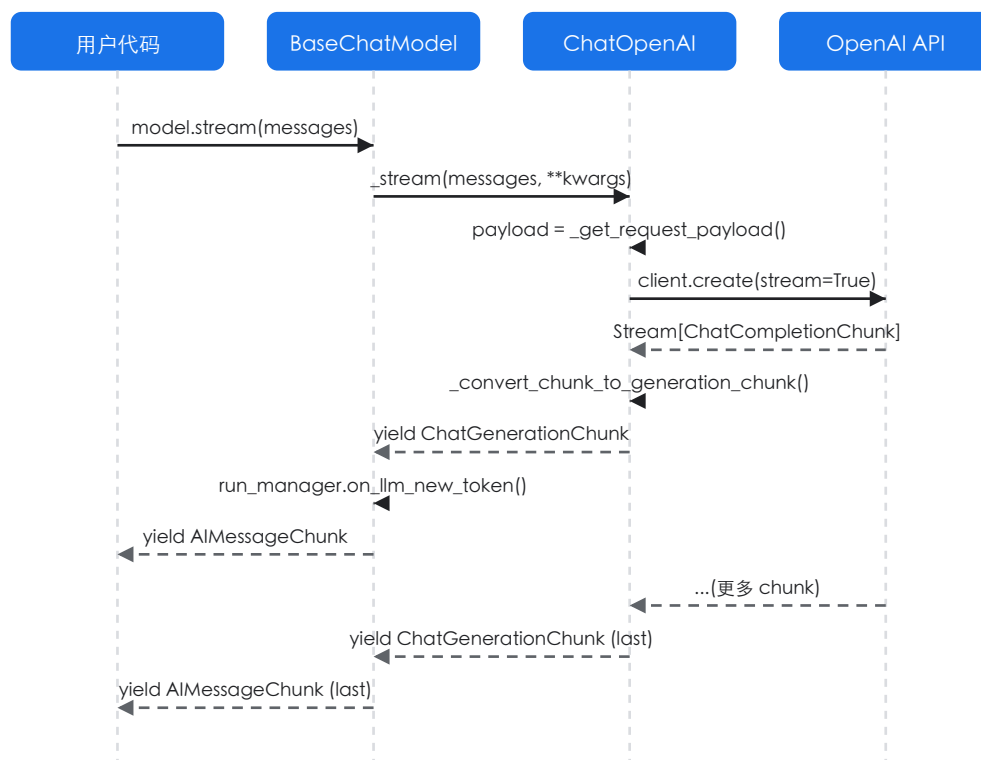


图 4-1: OpenAI Chat Completions 流式输出时序图

4.2 Anthropic 流式时序

Anthropic 使用原始事件流 (`RawMessageStreamEvent`)，事件类型更丰富，需要跟踪 `block_start_event` 来关联工具调用块的后续增量。

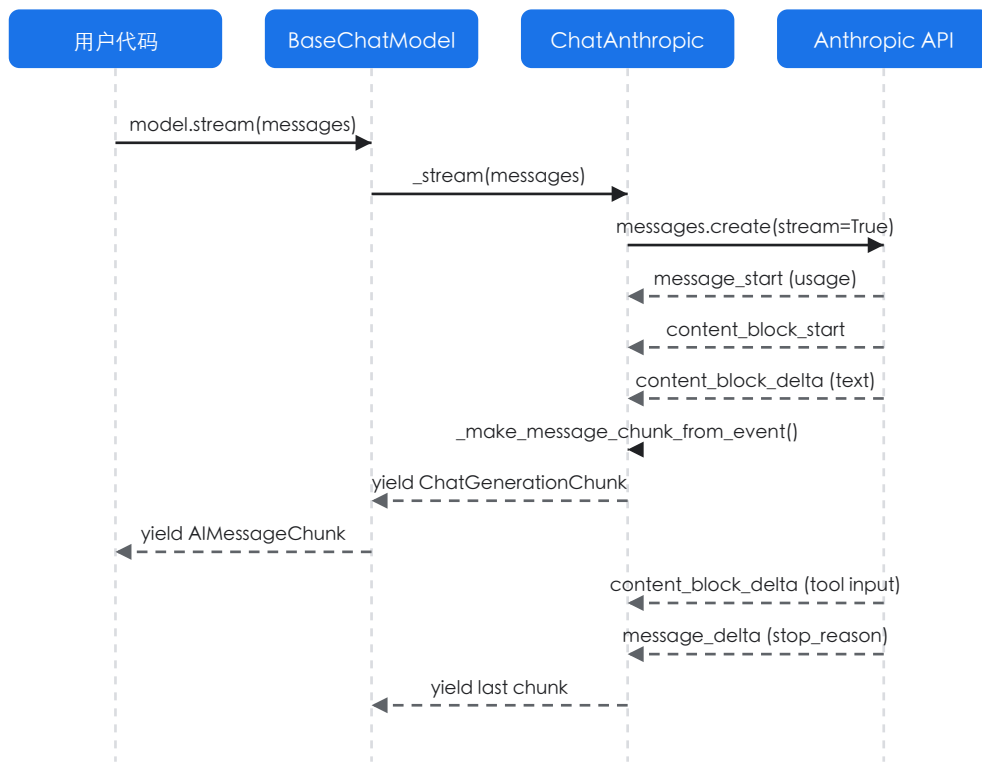


图 4-2: Anthropic 流式输出时序图

第五章 Tool Calling（工具调用）流程图

5.1 工具绑定与调用全流程

Tool Calling 是 LangChain 中最核心的能力之一，允许 LLM 调用外部工具。流程分为三个阶段：工具绑定 -> 模型推理 -> 工具执行。

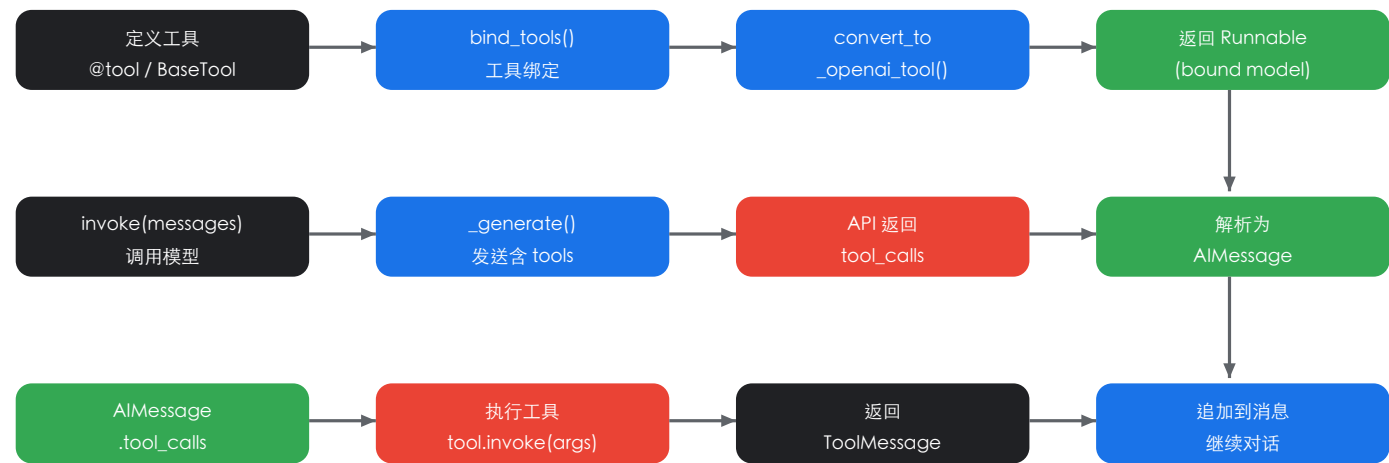


图 5-1: Tool Calling 完整流程图（三阶段）

5.2 工具格式转换对比

维度	OpenAI 格式	Anthropic 格式
外层结构	{"type":"function","function":{...}}	{"name":"...", "input_schema":{...}}
参数字段	parameters (JSON Schema)	input_schema (JSON Schema)
tool_choice	"auto" / "required" / {"type":"function",...}	"auto" / "any" / {"type":"tool","name":"..."}
内置工具	有限 (code_interpreter)	丰富 (text_editor, bash, web_search, mcp 等)
strict 模式	支持 (保证输出符合 schema)	支持

维度	OpenAI 格式	Anthropic 格式
并行调用控制	parallel_tool_calls 参数	disable_parallel_tool_use (tool_choice 上)

第六章 结构化输出 (Structured Output) 流程图

6.1 三种结构化输出方式

LangChain 通过 `with_structured_output()` 方法提供结构化输出能力，支持三种实现方式，各 Provider 默认方式不同。

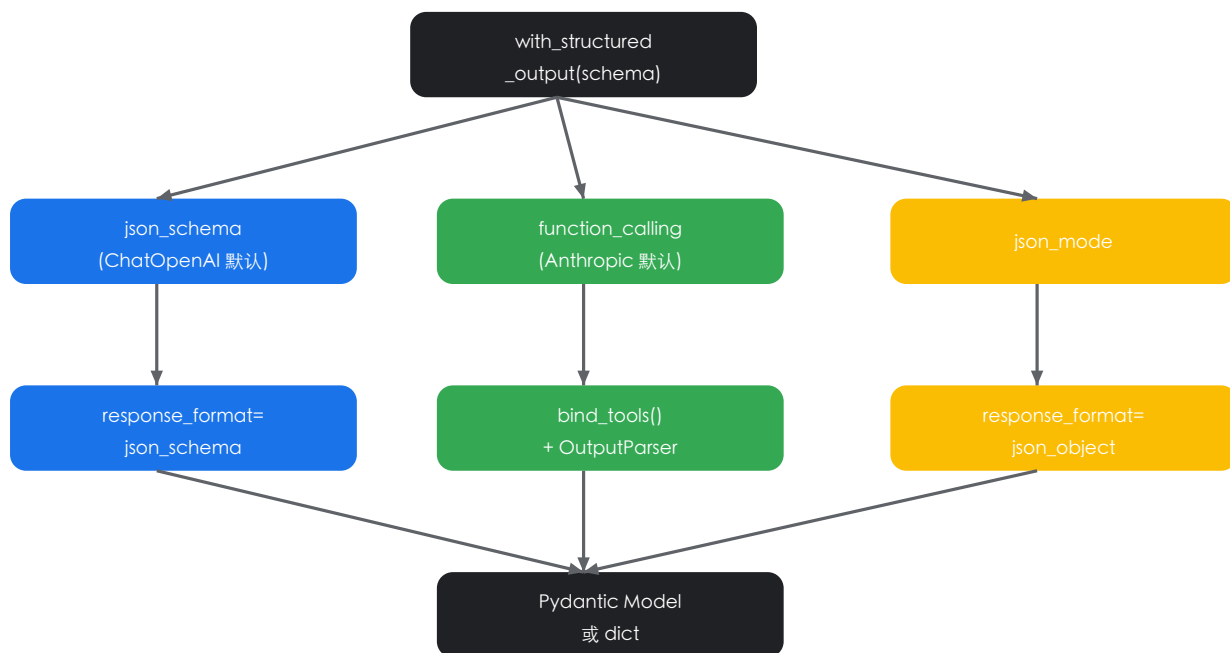


图 6-1: 结构化输出三种方式流程图

6.2 各 Provider 默认方式对比

Provider	默认 method	json_schema	function_calling	json_mode
ChatOpenAI	json_schema	支持 (原生 Structured Output API)	支持	支持
ChatAnthropic	function_calling	支持 (output_config.format)	支持	不支持 (重定向到 json_schema)
ChatOllama	json_schema	支持 (format=schema)	支持	支持 (format='json')
ChatGroq	function_calling	不支持	支持	支持
ChatMistralAI	function_calling	不支持	支持	不支持

第七章 Embeddings 嵌入处理流程图

7.1 OpenAI Embeddings 长文本处理流程

OpenAIEmbeddings 实现了智能的长文本处理策略：将超长文本自动分块嵌入，然后按 token 数加权平均合并，最后 L2 归一化。

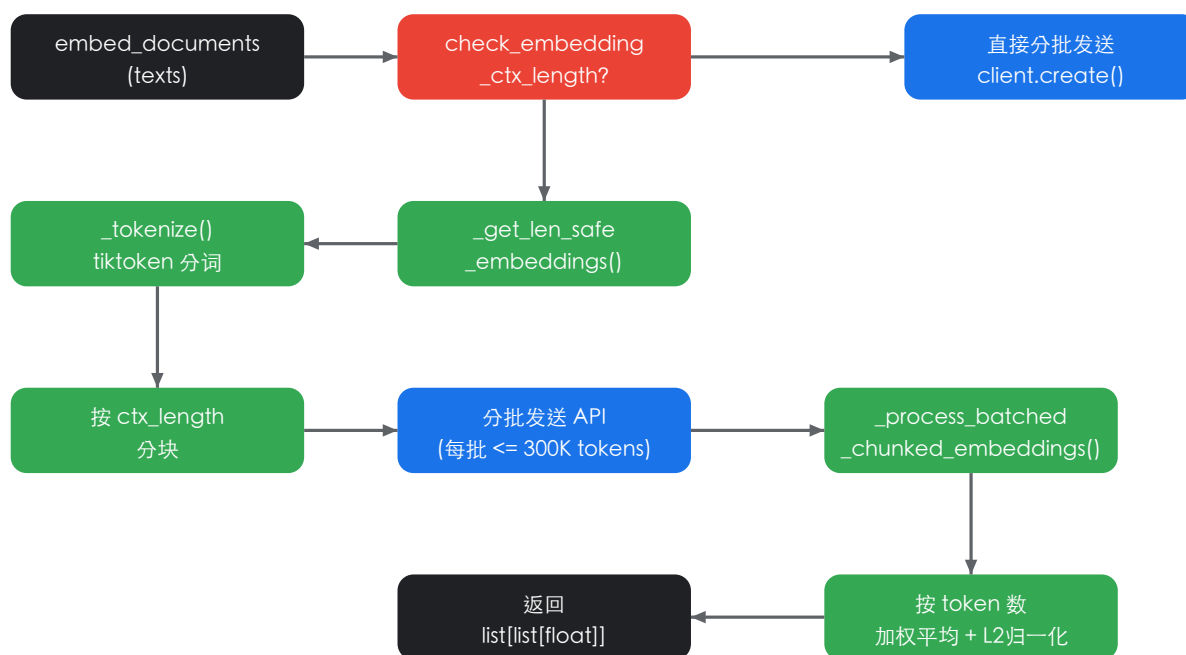


图 7-1: OpenAI Embeddings 长文本安全处理流程

7.2 各 Provider Embeddings 实现对比

Provider	核心类	默认模型	长文本处理	本地/远程
OpenAI	OpenAIEmbeddings	text-embedding-ada-002	自动分块+加权平均	远程 API
Ollama	OllamaEmbeddings	(用户指定)	原生处理	本地运行
HuggingFace	HuggingFaceEmbeddings	BAAI/bge-base-en-v1.5	sentence-transformers 处理	本地运行
HuggingFace	HuggingFaceEndpointEmbeddings	(用户指定)	API 端处理	远程 API
Fireworks	FireworksEmbeddings	(用户指定)	原生处理	远程 API
MistralAI	MistralAIEmbeddings	(用户指定)	原生处理	远程 API

第八章 向量存储 (VectorStore) 架构

8.1 VectorStore 检索流程

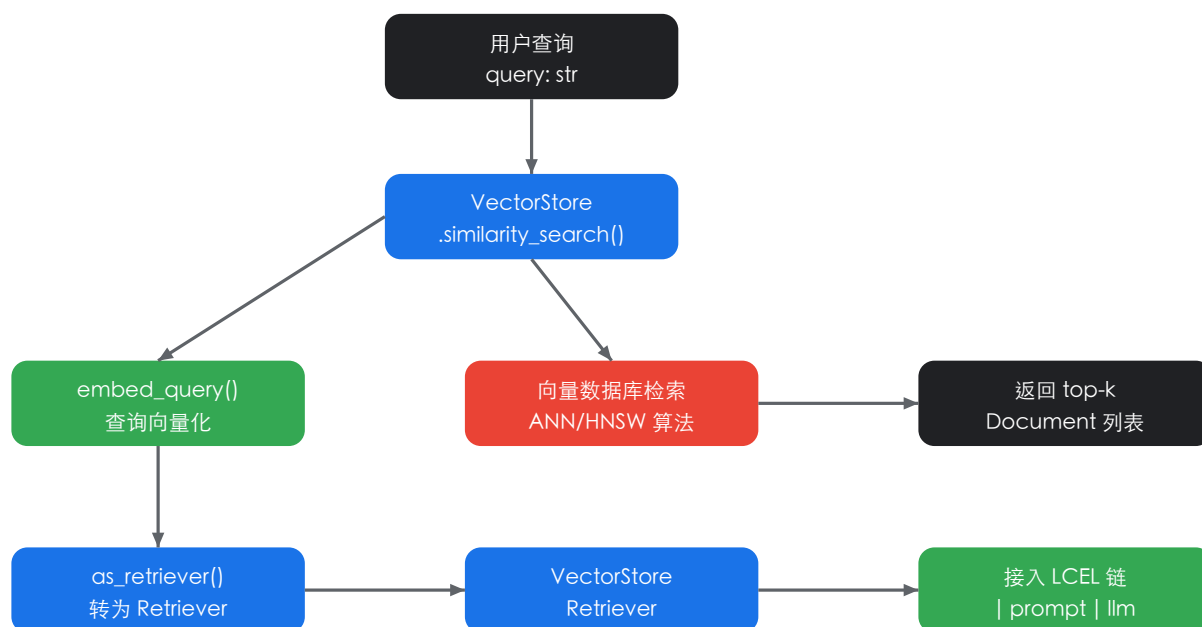


图 8-1: VectorStore 检索与 Retriever 桥接流程

8.2 Chroma vs Qdrant 对比

维度	Chroma	Qdrant
核心类	Chroma(VectorStore)	QdrantVectorStore(VectorStore)
存储后端	ChromaDB (本地/服务端)	Qdrant (本地/云端/内存)
初始化方式	from_documents() / 构造函数	from_documents() / from_existing_collection()
过滤能力	\$and/\$or/\$eq/\$ne 等	Filter + FieldCondition (类型丰富)
距离函数	cosine / l2 / ip	cosine / dot / euclid / manhattan
稀疏向量	不支持	支持 (SparseEmbedding + SparseRetrieval)
多向量	不支持	支持 (命名向量)
相似度+分数	similarity_search_with_score()	similarity_search_with_score()
MMR 搜索	max_marginal_relevance_search()	max_marginal_relevance_search()

第九章 OpenAI 集成包核心设计

9.1 包结构

```
langchain_openai/
├── chat_models/
│   ├── base.py          # BaseChatOpenAI + ChatOpenAI (~5000█)
│   ├── azure.py         # AzureChatOpenAI
│   ├── _client_utils.py # httpx ████
│   └── _compat.py        # ████████
├── embeddings/
│   ├── base.py          # OpenAIEmbeddings
│   └── azure.py         # AzureOpenAIEmbeddings
├── llms/
│   ├── base.py          # BaseOpenAI (Completions API)
│   └── azure.py         # AzureOpenAI
├── middleware/          # ████████
├── tools/               # @custom_tool ████
└── data/               # ██████ Profile
```

9.2 BaseChatOpenAI 核心字段

字段	类型	说明
model_name	str	模型名称，默认 gpt-3.5-turbo
temperature	float None	采样温度
max_tokens	int None	最大生成 token 数
streaming	bool	是否流式输出
use_responses_api	bool None	是否使用 Responses API
output_version	str None	AIMessage 输出格式版本 (v0/v1/responses)
reasoning_effort	str None	推理努力程度控制
disabled_params	dict None	禁用特定参数（兼容 OpenAI 兼容 API）
extra_body	Mapping None	透传自定义参数
service_tier	str None	服务层级 (auto/default/flex)

9.3 关键方法时序

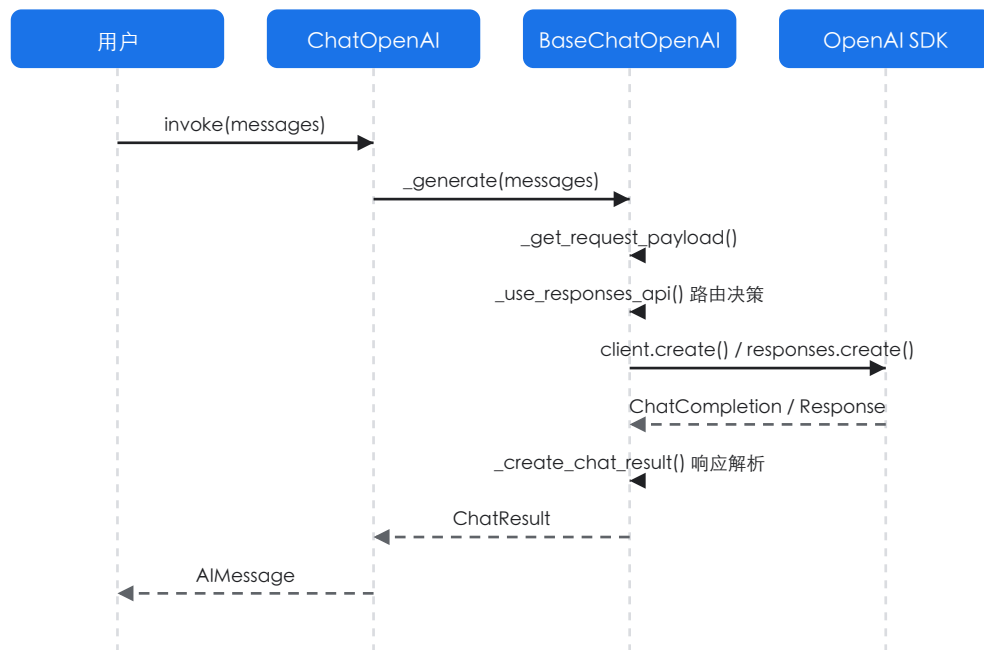


图 9-1: ChatOpenAI 核心调用时序

9.4 httpx 客户端缓存机制

OpenAI 包通过 `_client_utils.py` 实现了高效的连接管理：

- `_SyncHttpxClientWrapper`: 继承 `openai.DefaultHttpxClient`，添加 `__del__` 清理
- `@lru_cache`: 缓存 `httpx` 客户端实例，避免每个 `ChatOpenAI` 实例创建新连接
- 支持动态 API Key: 支持 `SecretStr`、同步/异步 callable 三种 API 密钥形式

第十章 Anthropic 集成包核心设计

10.1 包结构

```
langchain_anthropic/
chat_models.py    # ChatAnthropic (~2119█)
llms.py           # AnthropicLLM (████)
output_parsers.py # ToolsOutputParser
_compat.py        # v1 ████████
_client_utils.py  # httpx ██████
middleware/
  anthropic_tools.py # TextEditor + Memory ████
  bash.py            # Bash ██████
  file_search.py     # ████████
  prompt_caching.py  # ████████
data/               # █████ Profile
```

10.2 ChatAnthropic 核心设计

ChatAnthropic 直接继承 BaseChatModel（不经过中间层），完全独立实现了对 Anthropic Messages API 的集成。与 OpenAI 实现的最大区别在于消息格式和系统消息的处理方式。

10.2.1 消息格式转换

LangChain 消息	Anthropic 格式	说明
SystemMessage	独立 system 参数	Anthropic 的 system 是顶层参数，不在消息列表中
HumanMessage	role: "user"	直接映射
AIMessage	role: "assistant"	直接映射
ToolMessage	user 消息中的 tool_result block	合并到 HumanMessage 中（Anthropic 只有 user/assistant）
图片内容	source + media_type	与 OpenAI 的 image_url 格式不同

10.2.2 Beta 功能自动注入

ChatAnthropic 会在构建请求时自动扫描工具列表，根据工具类型自动追加所需的 Beta header。这使得用户无需手动管理 Beta 版本号。

工具类型	Beta Header	功能
code_execution_20250522	code-execution-2025-05-22	代码执行
mcp_toolset	mcp-client-2025-11-20	MCP 协议

工具类型	Beta Header	功能
computer_20251124	computer-use-2025-11-24	计算机使用
web_fetch_20250910	web-fetch-2025-09-10	网页抓取
memory_20250818	context-management-2025-06-27	记忆/上下文管理

10.2.3 Thinking（推理）模式

Anthropic 的 thinking 参数允许模型进行显式推理，流式传输中通过 thinking_delta 和 signature_delta 事件逐步返回推理过程。当 thinking 启用时，tool_choice 强制调用会被自动禁用（API 限制）。

10.2.4 Prompt Caching 中间件

AnthropicPromptCachingMiddleware 可以自动为消息添加缓存控制标记，减少重复 token 的消费。支持 5 分钟和 1 小时两种 TTL。

第十一章 其他 Partner 集成包设计

11.1 ChatOllama — 本地模型集成

ChatOllama 直接继承 BaseChatModel，通过 ollama Python SDK 与本地运行的 Ollama 服务通信。它是唯一一个面向本地模型的主要 Partner 包。

- 通过 ollama.Client / ollama.AsyncClient 与本地 Ollama 服务通信
- 支持 format='json' 的 JSON 模式输出
- 流式通过 ollama.Client.chat(stream=True) 实现
- OllamaEmbeddings 支持本地嵌入模型
- OllamaLLM 支持文本补全接口

11.2 ChatDeepSeek — OpenAI 兼容层复用

ChatDeepSeek 是最优雅的集成实现之一，它直接继承 BaseChatOpenAI，因为 DeepSeek API 与 OpenAI API 完全兼容。主要定制：

- 默认 base_url 设置为 https://api.deepseek.com
- API Key 环境变量改为 DEEPSEEK_API_KEY
- 特殊处理 reasoning_content (DeepSeek 的推理输出)
- 重写 _convert_chunk_to_generation_chunk() 处理推理流

11.3 ChatHuggingFace — 多后端支持

ChatHuggingFace 支持两种后端运行方式：

- HuggingFace Inference API: 远程 API 调用，通过 InferenceClient
- 本地 Pipeline: 使用 transformers 库本地运行模型
- 通过 tokenizer.apply_chat_template() 将消息转换为模型特定格式
- 自动检测模型是否支持 tool calling

11.4 ChatGroq — 高性能推理

ChatGroq 直接继承 BaseChatModel，完全独立实现。使用 Groq SDK 调用 Groq 的高性能推理 API。支持 tool calling 和流式输出。

11.5 ChatMistralAI — 独立实现

ChatMistralAI 直接继承 BaseChatModel，使用 Mistral AI SDK。支持 tool calling、流式输出、异步调用。

11.6 ChatFireworks — OpenAI 兼容

ChatFireworks 直接继承 BaseChatModel，但内部使用 OpenAI SDK 与 Fireworks AI 的 OpenAI 兼容端点通信。这是一种混合模式：继承独立基类但复用 OpenAI 客户端。

第十二章 跨 Provider 对比分析

12.1 实现策略对比

实现策略	代表 Provider	优势	劣势
继承 BaseChatOpenAI	DeepSeek	代码复用最大化，维护成本低	受限于 OpenAI 的接口设计
直接继承 BaseChatModel	Anthropic, Ollama, Groq	完全自主控制，可深度定制	代码量大，每家独立维护
独立实现 + OpenAI SDK	Fireworks	灵活性与复用的折中	两层抽象可能复杂

12.2 核心能力矩阵

能力	OpenAI	Anthropic	Ollama	DeepSeek	Groq	Mistral	Fireworks
Chat Model	Y	Y	Y	Y	Y	Y	Y
Completions LLM	Y	Y (废弃)	Y	-	-	-	Y
Embeddings	Y	-	Y	-	-	Y	Y
Tool Calling	Y	Y	Y	Y	Y	Y	Y
Structured Output	3种	2种	3种	继承 OpenAI	2种	1种	2种
Streaming	Y	Y	Y	Y	Y	Y	Y
异步原生	Y	Y	Y	Y	Y	Y	Y
推理/思考	reasoning_effort	thinking	-	reasoning_content	-	-	-
内置工具	有限	丰富	-	-	-	-	-
Middleware	审核	缓存+工具+Bash	-	-	-	-	-

12.3 消息格式差异

维度	OpenAI	Anthropic	Ollama
系统消息	role: 'system' 消息	独立 system 顶层参数	role: 'system' 消息

维度	OpenAI	Anthropic	Ollama
消息角色	system/user/assistant/tool	user/assistant (仅两种)	system/user/assistant /tool
ToolMessage	独立 tool 角色	合并到 user 的 tool_result block	独立 tool 角色
图片格式	image_url	source + media_type	images 列表 (base64)
工具调用	tool_calls 字段	content 中的 tool_use block	message.tool_calls

12.4 同步-异步桥接模式

所有 Partner 包都遵循 LangChain 的同步-异步桥接模式：默认情况下，所有异步方法（`ainvoke`, `astream`, `aembed_query` 等）通过 `run_in_executor` 自动将同步方法包装为异步版本。但大多数 Partner 包都重写了原生异步实现（如使用 `AsyncOpenAI`、`AsyncAnthropic` 客户端），以获得更好的性能。

— 文档结束 —

本文档基于 LangChain v0.1.16 源码分析生成，涵盖 `libs/partners` 目录下所有核心集成包。